

AEP-X — Operational Manual

Running, verifying, and troubleshooting the live reference implementation — with UML use case and sequence diagrams

Companion to Handoff, ADLC-Plan, Instructional-Manual, Microservices Guide, SOA-Architecture

Prepared 8 July 2026

Covers 14 core services · Connector Bus · 10 connector categories · 100 connectors

Contents

- | | |
|-----------------------------|--------------------------------------|
| 1. 1. System Overview | 5. 5. Smoke Test Procedure |
| 2. 2. UML Use Case Diagram | 6. 6. Troubleshooting Guide |
| 3. 3. UML Sequence Diagrams | 7. 7. Backup & Restore |
| 4. 4. Operations Runbook | 8. 8. Connector Operations |
| | 9. 9. Operational Definition of Done |

1. System Overview

The reference implementation runs as **32 containers** under one `docker-compose.yml` :

- **Tier 1 — Foundational (synchronous):** Identity, Trust, Registry, Memory, Cache (L0–L5), Discovery — plus the API Gateway that fronts everything.
- **Tier 2 — Orchestration (event-driven):** Workflow, Safety, Governance, Knowledge, Verification, Cost Optimiser, ML Integration.
- **SOA layer:** the Connector Bus plus 10 category connector services carrying **100 catalogued connectors** (4 specialized, 96 governed stubs — see [Connector-Catalogue.md](#)).
- **Infrastructure:** PostgreSQL (pgvector), Redis, Neo4j, Kafka (KRaft), Ollama (self-hosted model server).
- **Brain:** the self-healing/self-learning monitor — watches every core service's health, re-warms a cold Ollama, and runs a circuit breaker per connector learned from Governance's real audit history (§8.4).

Three invariants govern every runtime path (governance/book-of-laws.md): **Law 2** — no external call executes before a trust evaluation, enforced structurally at the Connector Bus; **Law 8** — every event is audited by the Governance Engine, which consumes every Kafka topic unconditionally; and the Brain's circuit breaker, which fails a repeatedly-unreliable connector fast rather than letting every caller rediscover the same timeout.

2. UML Use Case Diagram

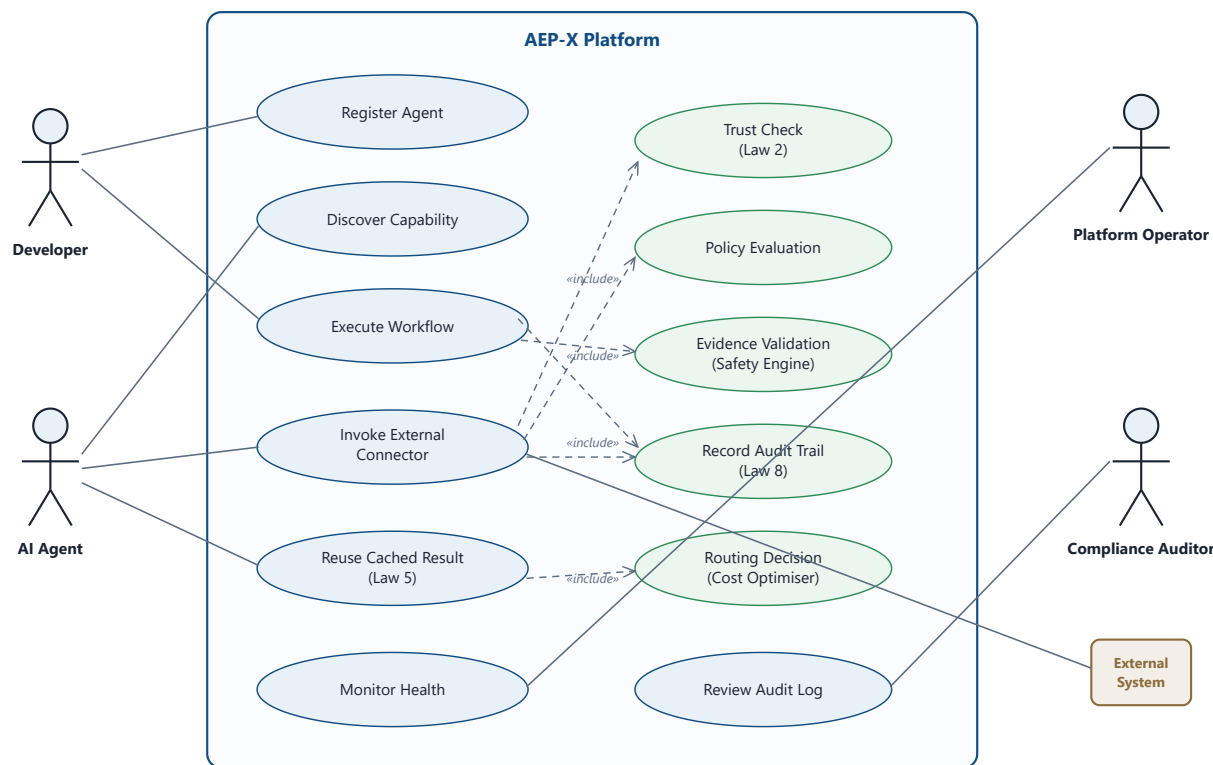


Fig. 1 — Use case diagram. Green use cases are governance-mandated inclusions: invoking any external connector *always* includes the trust check, policy evaluation, and audit recording — they are not optional paths.

3. UML Sequence Diagrams

The three load-bearing runtime flows, as verified live on 8 July 2026

3.1 Connector Bus mediation (SOA layer — Laws 2 & 8 enforced structurally)

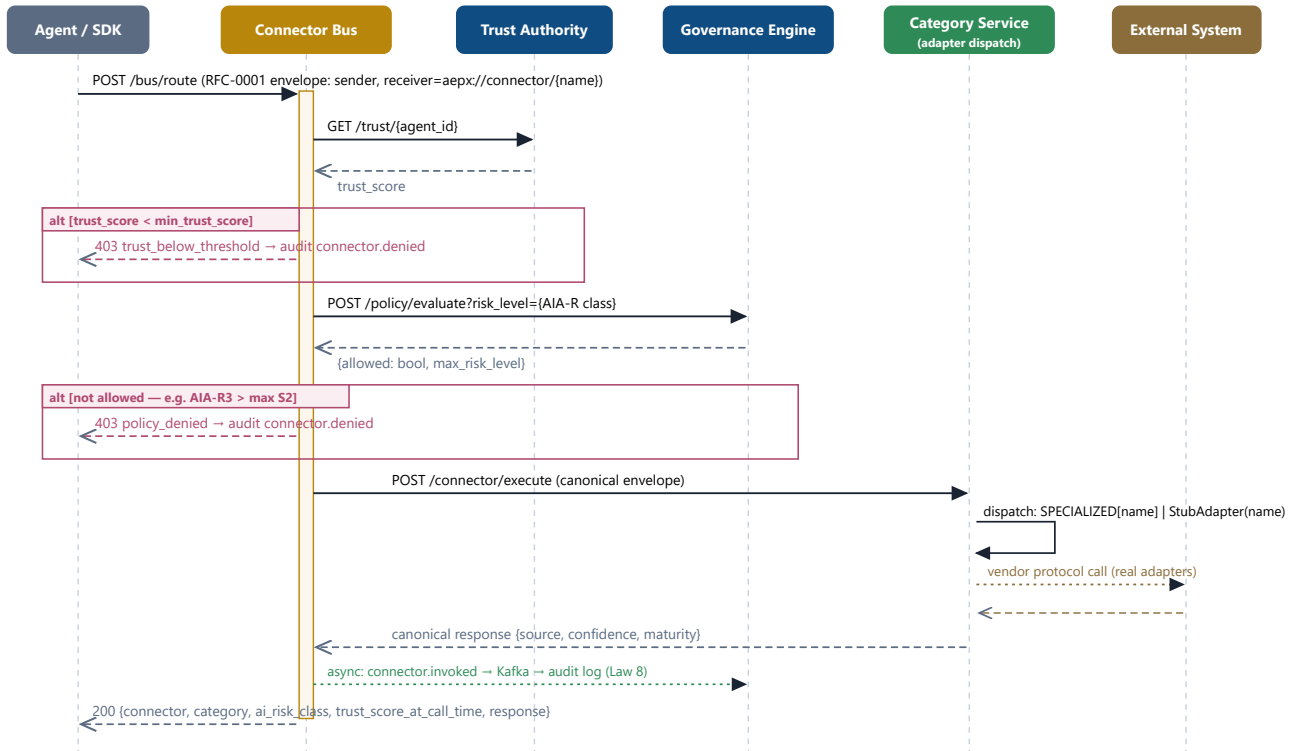


Fig. 2 — Connector Bus mediation. Both denial paths were exercised live: `opcua/gov-pay` (AIA-R3) exit at the policy gate; `sap/moodle` (min trust 60 vs. stub score 50) exit at the trust gate.

3.2 Workflow execution with event choreography (Tier 2 — async, Kafka)

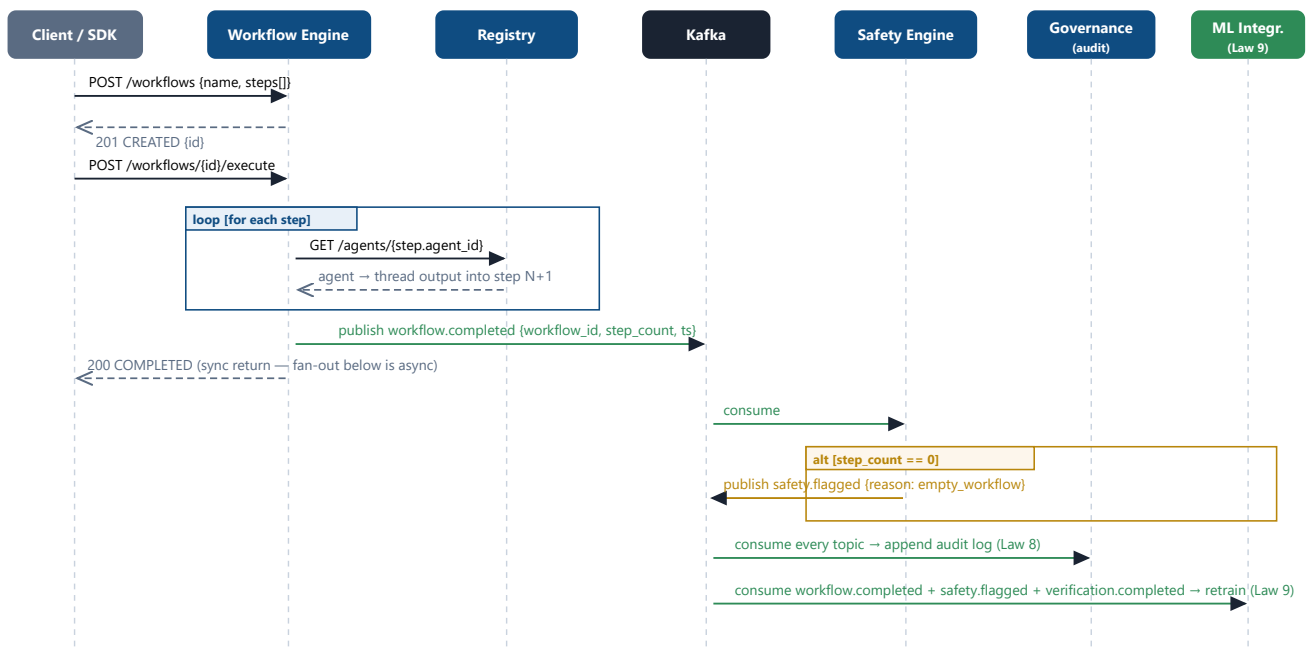


Fig. 3 — Event choreography. Governance is a passive listener — 100% audit coverage without any service having to call it. Consumers use reconnect loops (see §6, defect 3) so a broker restart never silently kills the chain again.

3.3 Cache-first routing (Law 5 — the LLM is the last resort)

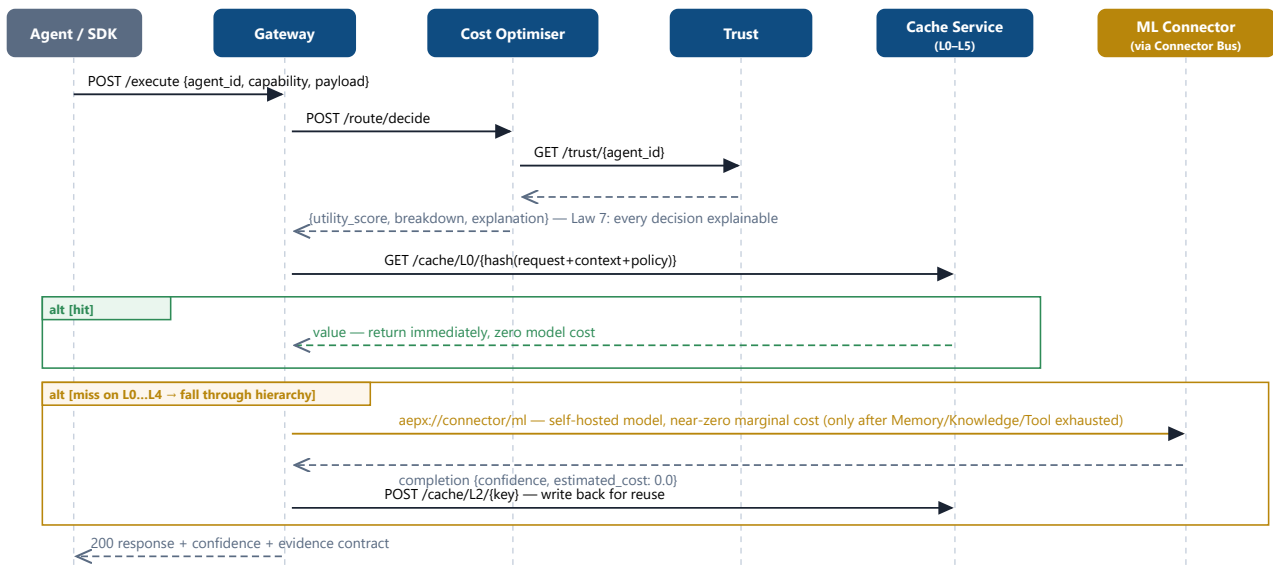


Fig. 4 — Cache-first execution (RFC-0005/0008). The Cost Optimiser's `/route/decide` is the single synchronous Tier-2 exception (<20 ms budget); L5 federation-cache writes additionally require a Governance policy check.

4. Operations Runbook

Start, stop, verify — the day-2 procedures for the running stack

4.1 Start / stop

```
# start everything (from the repo root)
docker compose up -d --build

# SOA layer only, after trust + governance are healthy (SOA-Architecture.md §5 ordering)
docker compose up -d trust governance
docker compose up -d --build connector-bus connector-enterprise connector-productivity \
  connector-devtools connector-aiplatform connector-data connector-messaging \
  connector-industrial connector-cloud connector-government connector-education

# stop (containers persist)           # stop and remove (data volumes preserved)
docker compose stop                  docker compose down

# full reset including data volumes - Postgres re-runs schemas/sql/*.sql on next start
docker compose down -v
```

4.2 Port map

Port	Service	Port	Service
8000	Gateway (aggregate <code>/health</code>)	8020	Connector Bus
8001	Identity	8021	connector-enterprise (14)
8002	Trust	8022	connector-aiplatform (14)
8003	Registry	8023	connector-devtools (11)
8004	Memory	8024	connector-productivity (11)
8005	Cache (L0–L5)	8025	connector-data (16)
8006	Discovery	8026	connector-messaging (8)
8007	Workflow Engine	8027	connector-industrial (8)
8008	Safety Engine	8028	connector-cloud (10)
8009	Governance (<code>/audit</code> , <code>/policy/evaluate</code>)	8029	connector-government (4)
8010	Knowledge	8030	connector-education (4)
8011	Verification	5432	PostgreSQL (pgvector/pgvector:pg16)
8012	Cost Optimiser	6379	Redis
8013	ML Integration	7474/7687	Neo4j (HTTP / Bolt)
8014	Brain (<code>/brain/status</code> , <code>/brain/circuit/<name></code>)	11434	Ollama (self-hosted model server)

8080	Console — the LLM/ML box web GUI (chat via Connector Bus; file/folder/image/video upload)	9092	Kafka (KRaft)
------	---	------	---------------

4.3 Health verification

```
curl http://localhost:8000/health      # 14 core services, aggregated
curl http://localhost:8020/health      # bus: expect "connector_count": 100, 10 categories
curl http://localhost:8009/audit       # audit log – grows with every workflow/safety event
curl http://localhost:8014/brain/status # every monitored service's up/down state
docker compose ps                     # 32 containers; postgres/redis/neo4j/ollama report (healthy)
```

5. Smoke Test Procedure

Run after any rebuild. Every step below was executed successfully against the live stack on 8 July 2026.


```
# 1. Register an agent
AGENT_ID=$(curl -s -X POST http://localhost:8003/agents \
  -H "Content-Type: application/json" -d '{"name":"smoke-agent"}' | jq -r .id)

# 2. Create + execute a workflow → expect status COMPLETED
WF_ID=$(curl -s -X POST http://localhost:8007/workflows -H "Content-Type: application/json" \
  -d '{"name":"smoke","steps":
[{"capability":"lesson_generation","agent_id":"$AGENT_ID"}]}') | jq -r .id)
curl -s -X POST http://localhost:8007/workflows/$WF_ID/execute | jq .status

# 3. Within ~2s the event must appear in the audit log (Law 8)
curl -s http://localhost:8009/audit | jq '.[1].topic' # → "workflow.completed"

# 4. Connector Bus – specialized adapter (expect Jane Doe)
curl -s -X POST http://localhost:8020/bus/route -H "Content-Type: application/json" \
  -d '{"sender":"aepx://agent/'$AGENT_ID',"receiver":"aepx://connector/salesforce","payload":
{"op":"lookup_contact"}}' | jq .response.result.name

# 5. Connector Bus – governance gates (expect HTTP 403 on both)
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8020/bus/route \
  -H "Content-Type: application/json" \
  -d '{"sender":"aepx://agent/x","receiver":"aepx://connector/opcua","payload":{}}' # policy
gate
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8020/bus/route \
  -H "Content-Type: application/json" \
  -d '{"sender":"aepx://agent/x","receiver":"aepx://connector/sap","payload":{}}' # trust
gate

# 6. Cost Optimiser – explainable routing decision (Law 7)
curl -s -X POST http://localhost:8012/route/decide -H "Content-Type: application/json" \
  -d
'{"agent_id":"'$AGENT_ID',"capability":"x","candidate_route":"cache","estimated_latency_ms":5
}' | jq .explanation
```

Pass criteria: step 2 returns COMPLETED; step 3 shows the event within 2 seconds; step 4 returns Jane Doe; step 5 returns 403 twice; step 6 returns an explanation string containing the utility score.

6. Troubleshooting Guide

The first three entries are real defects found and fixed during the 7–8 July deployment — kept here because their symptoms will recur in any fresh environment that repeats the original configuration.

Symptom	Root cause	Fix
<code>uvicorn:</code> <code>executable file</code> <code>not found in</code> <code>\$PATH</code> at container start; image builds "succeed" but pip layers are ~4 kB	Disk full during build — containerd writes truncated layers silently (<code>input/output</code> <code>error</code> in daemon logs)	Free disk space (Docker's VHDX lives under <code>AppData\Local\Docker</code>), then <code>docker compose build</code> <code>--no-cache</code>
Postgres exits (3): <code>extension</code> <code>"vector" is not</code> <code>available</code>	Stock <code>postgres:16</code> doesn't ship pgvector, which <code>002_v2_additions.sql</code> requires	Use <code>pgvector/pgvector:pg16</code> (already in docker- compose.yml). If a stale anonymous volume shows <i>collation mismatch</i> warnings: <code>docker compose rm -sfv</code> <code>postgres</code> && <code>docker compose up -d postgres</code>
Workflows complete but <code>GET</code> <code>/audit</code> stays empty; Kafka topic exists but has no consumers	Consumer/producer created once at import lost the startup race against the broker and died silently (no retry)	Fixed in source: reconnect loops (Governance, Safety) + lazy producers (Workflow, Safety). If it recurs: <code>docker</code> <code>compose restart workflow safety governance ml-</code> <code>integration</code> and check only <code>MainThread</code> isn't alone via <code>docker exec ... python -c "import threading;</code> <code>print(threading.enumerate())"</code>
Kafka exits at preflight: <code>Cluster ID</code> <code>string ... is not</code> <code>a valid UUID</code>	KRaft requires a base64- encoded 16-byte UUID (22 chars), not an arbitrary string	Generate one: <code>python -c "import uuid,base64;</code> <code>print(base64.urlsafe_b64encode(uuid.uuid4().bytes</code> <code>).decode().rstrip('='))"</code> → set as <code>CLUSTER_ID</code>
Bus returns <code>403</code> <code>trust_below_thr</code> <code>eshold</code>	Agent trust (stub: 50) below the connector's <code>min_trust_score</code> (e.g. SAP/education: 60)	Working as designed (Law 2). Unlocks when Trust Authority implements the RFC-0002 5-component formula — or is a per-connector governance decision in the catalogue
Bus returns <code>403</code> <code>policy_denied</code> for industrial/govern ment connectors	AIA-R3 exceeds Governance's seed <code>max_risk_level: S2</code>	Working as designed. Raising the ceiling is an explicit governance decision (<code>_POLICIES</code> in governance service), never a code workaround
Bus <code>/health</code> shows <code>connector_count</code> <code>: 0</code>	<code>catalogue.json</code> not mounted at <code>/app/catalogue.json</code>	Check the <code>volumes:</code> entry for connector-bus and every connector-* service in docker-compose.yml
Console shows "🚨 Could not reach the	Docker Desktop's daemon has wedged — on memory- constrained hosts (<8 GB RAM)	Restart the daemon: stop all <code>*docker*</code> processes, <code>wsl</code> <code>--shutdown</code> , relaunch Docker Desktop, wait ~40s, then <code>docker compose ps</code> — every service has <code>restart:</code>

<code>platform";</code> <code>docker</code> <code>version / docker</code> <code>ps</code> hang or return a 500 from the Linux engine pipe	this recurs under load once ~30 containers plus a loaded model are running, especially if Docker Desktop's Kubernetes feature is also enabled (it adds ~18 unused system containers: etcd, kube-apiserver, coredns, etc.)	<code>unless-stopped</code> so the whole stack self-heals with no further commands. If this recurs, disable Kubernetes in Docker Desktop Settings → Kubernetes (or set <code>"KubernetesEnabled": false</code> in <code>%APPDATA%\Docker\settings-store.json</code> while Docker Desktop is stopped) — this alone cut a live chat response from ~20s to ~6s on a 7.7 GB-RAM host by freeing an entire unused control-plane's worth of memory.
First chat reply after any Ollama restart returns <code>maturity: specialized_degraded</code> with a "model unreachable" message	Cold start — Ollama must load the model into RAM before its first inference, which can exceed the adapter's timeout (25s) on a constrained host	Working as designed (graceful degradation, not a bug). Warm it manually: <code>curl -X POST http://localhost:11434/api/generate -d '{"model":"llama3.2:1b", "prompt":"hi", "stream":false}'</code>, then retry — subsequent calls return in 3–7s
Console chat times out or shows "Could not reach the platform" intermittently, seemingly at random, even though <code>docker compose ps</code> shows everything healthy	Ollama's default <code>OLLAMA_KEEP_ALIVE</code> (5 minutes) unloads the model between chats on a normally-paced conversation, forcing a cold reload on the next message — measured as slow as 88s on this box, well past the aiplatform connector's 25s timeout	Fixed in <code>docker-compose.yml</code>: <code>OLLAMA_KEEP_ALIVE=-1</code> on the <code>ollama</code> service keeps the model loaded indefinitely once warm. If this regresses, confirm with <code>docker exec aep-x-ollama-1 env grep KEEP_ALIVE</code>

7. Backup & Restore

- **What to back up:** the repo itself is the system of record — all images rebuild from Dockerfiles; base images re-pull from Docker Hub. Runtime state that matters lives in Postgres volumes only once real pilot data exists.
- **Current backups (8 July 2026):** `OneDrive\Backups\AEP-X\aepx-core-full-2026-07-08.tar.gz` (complete repo) and `Docker-Backup-2026-07-08\` (build definitions + restore README). Cloud sync engages when the OneDrive client is signed in.
- **Restore:** extract the tarball → `docker compose up --build`. Postgres re-initialises from `schemas/sql/` on first boot with an empty volume.

- **Do not back up Docker's binary image data** (`docker_data.vhdx`) — it is ~20 GB of rebuildable artifacts, and images built during a disk-full incident are corrupted anyway (see §6, defect 1).

8. Connector Operations

8.1 Promoting a stub to a real integration

1. Write an adapter class in the category service's `app/adapters.py` implementing `execute(payload) → dict`, and register it: `SPECIALIZED["notion"] = NotionAdapter()`.
2. Store credentials in Vault/env — never in the catalogue or code.
3. Update the connector's `maturity` to `specialized` in `connectors/catalogue.json` (single source of truth) and regenerate the SQL/docs.
4. For AIA-R2+: record the assurance-tier sign-off in `governance/decisions/` *before* first live traffic (SOA-Architecture.md §4).
5. `docker compose up -d --build connector-<category>` — nothing else redeploys.

8.2 Adding a brand-new connector

Add one entry to `connectors/catalogue.json` and restart the bus + owning category service. It is immediately routable as a governed stub — trust-checked, policy-gated, audited — with no code change. That is the point of the data-driven routing table.

8.3 The trust and policy gates (do not bypass)

A 403 from the bus is a **feature**. The trust gate (Law 2) and the policy ceiling protect exactly the connectors — industrial control systems, government services, HR/finance platforms — where an ungoverned agent call does real-world damage. Raise an agent's trust or the risk ceiling through governance records, never by editing the check out of the bus.

8.4 The third gate: the Brain's circuit breaker (also do not bypass)

A 503 from the bus is the same kind of feature, for a different reason. Unlike trust/policy (fixed rules, evaluated per call), the circuit breaker is **learned** — `services/brain` derives each connector's reliability from Governance's real audit history and opens the circuit after `CIRCUIT_FAILURE_THRESHOLD` (5) consecutive failures, closing it again only after a cooldown trial succeeds. It is deliberately HTTP-only (no `docker.sock`, which would be root-equivalent host control with no real justification for a health monitor) and fails **open** — if the Brain itself is unreachable, the bus proceeds as if the circuit were closed rather than blocking normal traffic. Check `GET /brain/circuit/<name>` before assuming a connector is broken; do not silence a 503 by removing the bus's call to the Brain.

9. Operational Definition of Done

- All 32 containers up; `postgres`, `redis`, `neo4j`, `ollama` report `(healthy)`.
- Gateway `/health`: all 14 services `ok`. Bus `/health`: `connector_count: 100`, 10 categories. Brain `/health`: `monitoring: 16`.
- Smoke test (\$5) passes all six criteria, including both 403 governance gates. v1 test environment (`tests/v1/run_smoke_test.sh`) passes all 6 checks in isolation.
- A workflow execution appears in `GET /audit` within 2 seconds (choreography chain alive); a Brain circuit-breaker trip appears there too (Law 8 covers self-healing actions, not just application events).
- Backups current (\$7) and the restore path tested at least once per release.